FamScript Visual Builder — Product Specification Sheet

Based on the uploaded FamScript Specification v1.0.

---

1. Project Overview

Product Name

FamScript Visual Builder

Product Type

Browser-based visual genealogy/family-tree compiler and renderer.

Core Goal

Allow users to:

1. Write simplified FamScript syntax
2. Convert it into valid Mermaid code
3. Render family trees visually using Mermaid
4. Import/export Mermaid and JSON data
5. Export rendered trees as SVG, PNG, JPG, and Markdown

The system follows the architecture and validation philosophy defined in the uploaded FamScript rulebook/specification.

---

2. Tech Stack

Frontend Stack

Layer	Technology
Structure	HTML5

Styling	CSS3
Logic	Vanilla JavaScript
Rendering Engine	Mermaid.js
Tree Preview	Mermaid Live Rendering
Export Engine	SVG + Canvas API
Data Storage	JSON
File Handling	FileReader API + Blob API

---

3. System Architecture

User Input (FamScript)
↓
FamScript Parser
↓
Validation Engine
↓
Mermaid Code Generator
↓
Mermaid Renderer
↓
SVG Tree Output
↓
Export Engine

Architecture follows the uploaded specification flow.

---

4. Core Features

4.1 FamScript Input Editor

Description

Main text editor where users write FamScript syntax.

Features

- Syntax highlighting
- Live validation
- Auto-indentation
- Line numbering
- Real-time compile status

Example Input

```
PERSON GROOM_SLF_001
NAME Arjun Kumar
AGE 28
GENDER M
GEN G0

PERSON BRIDE_SLF_001
NAME Priya Singh
AGE 25
GENDER F
GEN G0

RELATION GROOM_SLF_001 MARRIED_TO BRIDE_SLF_001
```

---

5. FamScript Parser Engine

Responsibilities

The parser converts simplified FamScript into structured JSON.

Parser Output Example

```
{
 "nodes": [
   {
     "id": "GROOM_SLF_001",
     "name": "Arjun Kumar",
     "age": 28,
     "gender": "M",
     "generation": "G0"
   }
 ],
 "relations": [
   {
     "from": "GROOM_SLF_001",
     "to": "BRIDE_SLF_001",
     "type": "MARRIED_TO"
```

```
  }
 ]
}
```

---

6. Mermaid Generator

Responsibilities

Convert parsed JSON into Mermaid graph syntax.

Generated Mermaid Example

The Mermaid syntax follows the FamScript node/relationship rules defined in the spec.

---

7. Mermaid Live Renderer

Renderer Engine

Use:

```
<script src="https://cdn.jsdelivr.net/npm/mermaid@10/dist/mermaid.min.js"></script>
```

Render Flow

```
mermaid.initialize({
 startOnLoad: false,
 theme: "default"
});

mermaid.render("tree", mermaidCode)
```

---

8. Validation Engine

Validation rules are based directly on the uploaded FamScript specification.

Required Validation Rules

Rule	Description
Age Rule	Parent must be ≥ child + 15
Generation Rule	Generations must match hierarchy
Gender Rule	Father = M, Mother = F
Duplicate Rule	No duplicate IDs
Circular Relation Rule	No cyclic ancestry
Marriage Rule	Spouses must share generation
Child Generation Rule	Child must be lower generation
Missing Data Rule	Required fields enforced

---

9. Mermaid Import System

Description

Users can paste existing Mermaid code.

Flow

Mermaid Code
↓
Mermaid Parser
↓
Internal JSON Model
↓
Tree Render

Supported

- graph TB
- subgraph
- relationship edges
- node metadata

---

10. JSON Import/Export

Export JSON

Example

```
{
 "projectName": "Kumar Family",
 "version": "1.0",
 "nodes": [],
 "relations": [],
 "settings": {
   "theme": "dark"
 }
}
```

Import JSON

The application reconstructs:

- nodes
- relationships
- layouts
- themes
- metadata

---

11. Markdown Export (.md)

Description

Export Mermaid code inside markdown fences.

Output

````
# Family Tree

```mermaid
graph TB
...
```
````

Use Cases

- GitHub README
- Obsidian
- Notion
- GitBook

---

12. Export System

Supported Formats

Format	Method
SVG	Direct Mermaid SVG
PNG	SVG → Canvas
JPG	Canvas conversion
Markdown	Blob file
JSON	Blob file

---

13. SVG Export Flow

Steps

Rendered SVG
↓
Serialize SVG
↓
Download .svg

JS Example

```
const svgData = new XMLSerializer().serializeToString(svgElement);
```

---

14. PNG/JPG Export Flow

Steps

SVG
↓
Canvas
↓
toDataURL()
↓
PNG/JPG Download

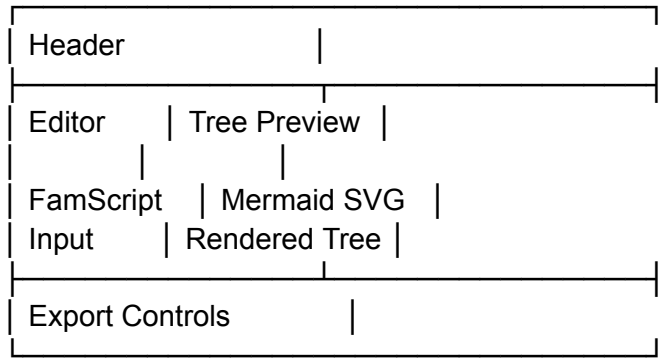
JS Example

canvas.toDataURL("image/png")
canvas.toDataURL("image/jpeg")

---

15. UI Layout

Application Layout



---

16. UI Components

Components List

Component	Purpose
Header	Branding + controls
Sidebar	File actions
Editor	FamScript input
Preview Panel	Mermaid tree

Console Panel	Validation logs
Export Toolbar	Downloads
Theme Selector	Mermaid themes

---

17. Mermaid Themes

Supported Themes

Theme
default
dark
forest
neutral
base

As defined in the FamScript playground system.

---

18. Folder Structure

```
project/
│
├── index.html
├── style.css
├── app.js
├── parser.js
├── validator.js
├── mermaidGenerator.js
├── export.js
├── importer.js
├── utils.js
│
├── assets/
│
└── data/
```

---

19. Main Modules

parser.js

Converts FamScript → JSON

validator.js

Runs validation rules

mermaidGenerator.js

JSON → Mermaid

importer.js

Mermaid/JSON → internal model

export.js

SVG/PNG/JPG/MD/JSON exports

---

20. Supported Relationship Types

Based on specification.

Type	Mermaid Edge
Parent	-->
Mother	-->
Marriage	---
Core Couple	==>
Step Relation	-.->

Cousin	-.->
Adoptive	-.->

---

21. Data Model

Node Model

```
{
 id: "",
 name: "",
 age: 0,
 gender: "",
 generation: "",
 status: ""
}
```

Relation Model

```
{
 from: "",
 to: "",
 type: ""
}
```

---

22. Rendering Rules

Following uploaded spec.

Rules

- Older generations appear above
- G0 centered
- Children below
- Straight hierarchy lines
- Minimal edge crossing
- Subgraph grouping by generation

---

23. Performance Goals

Metric	Target
Initial Render	< 1s
Max Nodes	10,000+
Live Compile Delay	< 100ms
Export Time	< 2s

---

24. Error Handling

Errors

Error	Message
Duplicate ID	"Duplicate node ID found"
Invalid Age	"Parent younger than child"
Invalid Generation	"Generation mismatch"
Circular Tree	"Circular ancestry detected"
Invalid Mermaid	"Mermaid parse failed"

---

25. Browser Support

Browser	Support
Chrome	Yes
Edge	Yes
Firefox	Yes
Brave	Yes
Safari	Partial

---

26. Future Enhancements

Planned Features

- Drag-and-drop tree editing
- AI-assisted relationship generation
- Automatic generation balancing
- Zoom/pan engine
- Database persistence
- Multi-family merging
- Live collaboration
- PDF export
- React version
- Offline desktop app

---

27. Final Product Workflow

FamScript Input
↓
Parse
↓
Validate
↓
Generate Mermaid
↓
Render Tree
↓
Export SVG / PNG / JPG / JSON / MD

---

28. Deliverables

Required Files

File	Purpose
index.html	Main UI
style.css	Styling
app.js	Main logic

parser.js	FamScript parser
validator.js	Validation
mermaidGenerator.js	Mermaid conversion
export.js	Export utilities
importer.js	Import system

---

29. Development Priority

Phase 1

- Editor
- Parser
- Mermaid rendering

Phase 2

- Validation engine
- Import/export

Phase 3

- Performance optimization
- Advanced layouts

Phase 4

- AI-assisted generation
- Collaboration tools

---

30. Final Vision

FamScript aims to become:

> “The Mermaid-native declarative standard for family”